

GlovoApp Backend (Flask + MariaDB)

Това е сървърна и конзолна реализация на малък Glovo-подобен проект с три роли: клиент, куриер и администратор. Проектът използва Python (Flask) и база от данни MariaDB (MySQL).

Използвани библиотеки

- Flask — за създаване на HTTP API сървър
- Flask-CORS — за позволяване на крос-домейн заявки
- mysql-connector-python — за връзка с MariaDB
- requests — за изпращане на HTTP заявки от CLI приложението

Инсталация:

```
pip install -r requirements.txt
```

Структура на проекта

— app.py	# Flask API сървър
— cli.py	# Конзолно приложение за взаимодействие с API-то
— config.py	# Настройки за база от данни
— db.py	# Логика за работа с база (използвана от API-то)
— requirements.txt	# Съдържа всички библиотеки

Как работи проектът

- **Сървърна част (app.py):**
 - Приема заявки чрез HTTP (POST, GET)
 - Обработва регистрация, вход, създаване на ресторанти, поръчки и др.
 - Всички заявки отиват към db.py, който комуникира с MariaDB чрез mysql.connector
- **CLI част (cli.py):**
 - Взаимодейства с потребителя в терминала
 - Изпраща HTTP заявки към Flask сървъра чрез requests
 - Предоставя менюта според ролята на потребителя (customer, courier, admin)
- **Файл с базова конфигурация (config.py):**
 - Съдържа детайли за свързване с MySQL сървъра
- **База данни (db.py):**
 - Съдържа всички функции за работа с базата от данни: добавяне на потребители, създаване на ресторанти, менюта, поръчки и т.н.
 - Използва параметризирани заявки за защита срещу SQL инжекции

Стартиране на проекта

1. Настрой база данни

Създай база с име glovo_app и таблици:

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE,  
    password VARCHAR(100),  
    role ENUM('customer', 'courier', 'admin')  
);  
  
CREATE TABLE restaurants (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    created_by INT,  
    FOREIGN KEY (created_by) REFERENCES users(id)  
);  
  
CREATE TABLE menu_items (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    price FLOAT,  
    restaurant_id INT,  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(id)  
);  
  
CREATE TABLE orders (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id INT,  
    restaurant_id INT,  
    item_id INT,  
    quantity INT,  
    status ENUM('pending', 'completed') DEFAULT 'pending',  
    courier_id INT,  
    FOREIGN KEY (customer_id) REFERENCES users(id),  
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(id),  
    FOREIGN KEY (item_id) REFERENCES menu_items(id),  
    FOREIGN KEY (courier_id) REFERENCES users(id)  
);
```

2. Конфигурация на връзката

Провери config.py:

```
DB_CONFIG = {  
    'host': 'localhost',  
    'user': 'root',  
    'password': 'root',  
    'database': 'glovo_app'  
}
```

3. Стартирай Flask сървъра

```
python app.py
```

4. Стартирай CLI клиента

```
python cli.py
```

API Крайни точки (Endpoints)

Метод	Път	Описание
POST	/register	Регистрация на потребител
POST	/login	Вход на потребител
GET	/restaurants	Извличане на списък с ресторанти
GET	/menu/	Меню за конкретен ресторант
POST	/order	Създаване на поръчка
GET	/courier/orders	Списък с чакащи поръчки за куриери
POST	/courier/complete	Завършване на поръчка от куриер
POST	/admin/restaurant	Добавяне на ресторант от администратор
POST	/admin/item	Добавяне на артикул в меню

Роли в системата

- **Клиент (customer):**
 - Вижда ресторанти и менюта
 - Прави поръчки
- **Куриер (courier):**
 - Вижда всички чакащи поръчки
 - Завършва поръчки (назначаване се сам автоматично)
- **Администратор (admin):**
 - Добавя ресторанти и артикули в тях

Примерни заявки (Postman)

Регистрация

Метод: POST

URL: <http://localhost:5000/register>

Body (raw JSON):

```
{  
  "username": "test",  
  "password": "1234",  
}
```

```
"role": "customer"
}
```

Вход

Метод: POST

URL: <http://localhost:5000/login>

Body (raw JSON):

```
{
  "username": "test",
  "password": "1234"
}
```

Вземи ресторанти

Метод: GET

URL: <http://localhost:5000/restaurants>

Вземи меню на ресторант (напр. с ID 1)

Метод: GET

URL: <http://localhost:5000/menu/1>

Направи поръчка

Метод: POST

URL: <http://localhost:5000/order>

Body (raw JSON):

```
{
  "customer_id": 1,
  "restaurant_id": 1,
  "item_id": 2,
  "quantity": 1
}
```

Препоръки за тестване

- Стартирай първо сървъра (app.py), после CLI клиента (cli.py)
- Може да тестваш API-то самостоятелно чрез Postman/curl
- За всеки потребител, използвай различна роля и провери функционалността ѝ